

Lecture 9: Feed-Forward Networks

Instructor: Stephanie Schoch

CSCI 680: Natural Language Processing
Department of Computer Science, College of William & Mary
September 23, 2026



Lecture Outline

- Quiz Recap
- Feed-Forward Networks

Review: Perplexity

- **Perplexity** is the inverse probability of the test set, normalized by the number of words

$$PP(W) = P(w_1 w_2 \dots w_N)^{-\frac{1}{N}}$$

3. Two language models are evaluated on the **same test set**. Model A has perplexity 62 and Model B has perplexity 91. Which statement is justified?
- A. Model A has lower average negative log-likelihood per token on the test set.
 - B. Model A assigns higher probability than Model B to every token in the test set.
 - C. Model A must also have lower loss on its training data.
 - D. Model A is guaranteed to have lower perplexity than Model B on any other test set.

Review: Perplexity

- **Perplexity** is the inverse probability of the test set, normalized by the number of words

$$PP(W) = P(w_1 w_2 \dots w_N)^{-\frac{1}{N}}$$

$$P(\text{test}) = \prod_{i=1}^N P(w_i \mid w_1, \dots, w_{i-1})$$

$$\text{NLL} = -\log P(\text{test}) = -\sum_{i=1}^N \log P(w_i \mid w_1, \dots, w_{i-1})$$

$$\text{Average NLL} = -\frac{1}{N} \log P(\text{test}) = -\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid w_1, \dots, w_{i-1})$$

$$PP = \exp \left(-\frac{1}{N} \log P(\text{test}) \right) \quad PP = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid w_1, \dots, w_{i-1}) \right)$$

Review: Precision & Recall

7. A binary classifier estimates

$$\hat{y} = P(y = 1 \mid x).$$

On a test set of 10 examples, its outputs are:

True label (y)	Estimated probabilities ($\hat{y}$)
1	0.90, 0.70, 0.40, 0.20
0	0.80, 0.65, 0.45, 0.35, 0.15, 0.05

Using a decision threshold of $\hat{y} \geq 0.50$, what are the **precision and recall** for the positive class?

- A. Precision = 0.67, Recall = 0.50
- B. Precision = 0.50, Recall = 0.67
- C. Precision = 0.50, Recall = 0.50
- D. Precision = 0.67, Recall = 0.67

Review: Precision & Recall

7. A binary classifier estimates

$$\hat{y} = P(y = 1 \mid x).$$

On a test set of 10 examples, its outputs are:

True label (y)		Estimated probabilities ($\hat{y}$)	
1	TP	0.90, 0.70,	0.40, 0.20
0	FP	0.80, 0.65,	0.45, 0.35, 0.15, 0.05

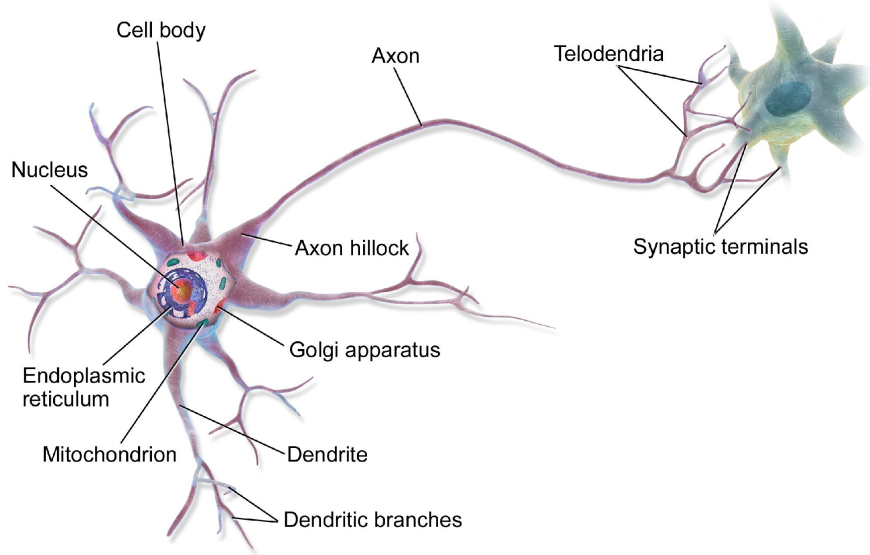
Using a decision threshold of $\hat{y} \geq 0.50$, what are the **precision** and **recall** for the positive class?

- A. Precision = 0.67, Recall = 0.50
- B. Precision = 0.50, Recall = 0.67
- C. Precision = 0.50, Recall = 0.50
- D. Precision = 0.67, Recall = 0.67

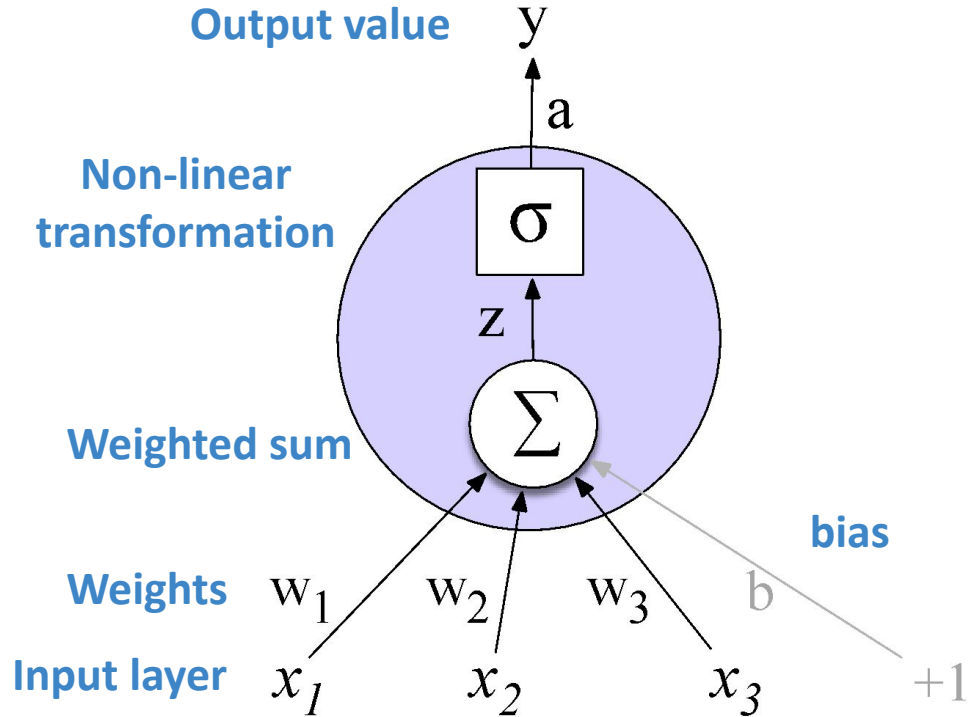
$$\text{precision} = \frac{\text{tp}}{\text{tp} + \text{fp}}$$

$$\text{recall} = \frac{\text{tp}}{\text{tp} + \text{fn}}$$

Biological vs. Neural Network Units

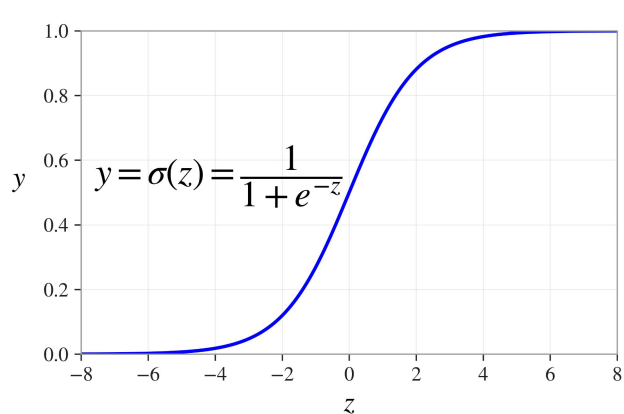


A neuron in the brain

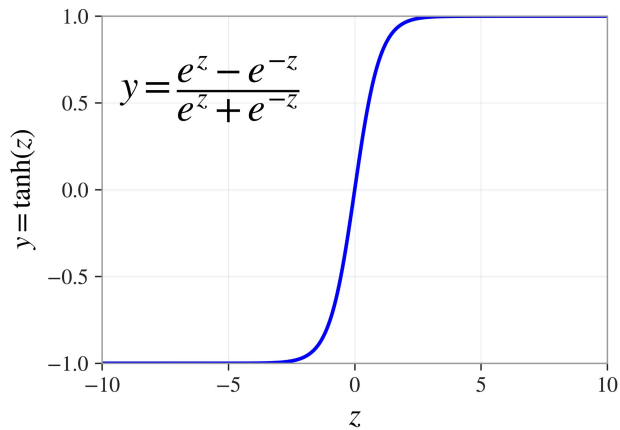


A neural unit

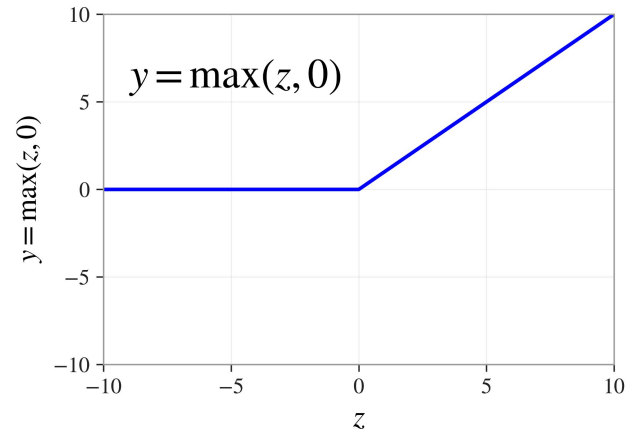
Non-Linear Activation Functions



sigmoid



tanh



relu
(Rectified Linear Unit)

Let's Consider XOR

Minsky and Papert (1969)

Can neural units compute simple functions of input?

AND			OR			XOR		
x1	x2	y	x1	x2	y	x1	x2	y
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

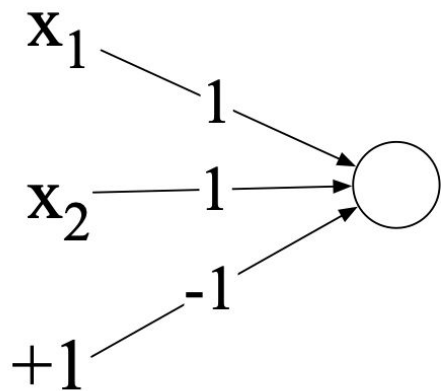
Perceptrons

- A very simple neural unit
 - Binary output (0 or 1)
 - No non-linear activation function

$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases}$$

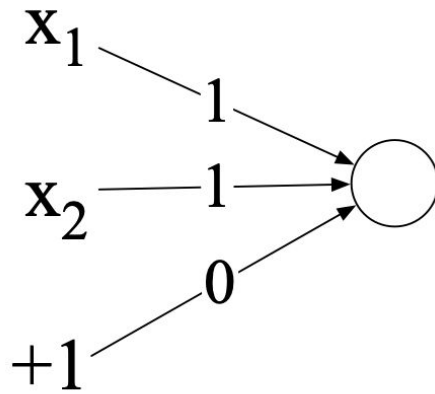
Easy to build AND or OR with perceptrons

$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases}$$



AND

AND		
x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1



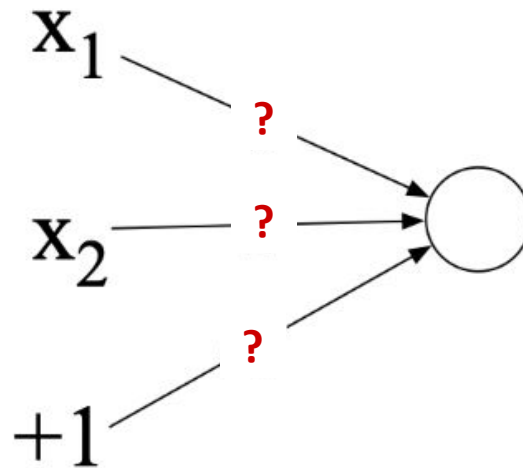
OR

OR		
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	1

Can We Capture XOR With Perceptrons?

$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases}$$

XOR		
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0



Why? Perceptrons are linear classifiers

Perceptron equation given x_1 and x_2 , is the equation of a line

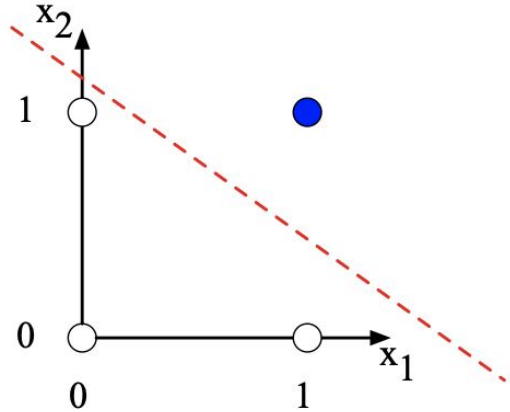
$$w_1x_1 + w_2x_2 + b = 0$$

(in standard linear format: $x_2 = (-w_1/w_2)x_1 + (-b/w_2)$)

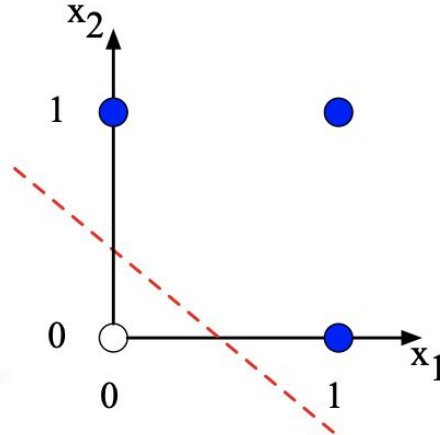
This line acts as a **decision boundary**

- 0 if input is on one side of the line
- 1 if on the other side of the line

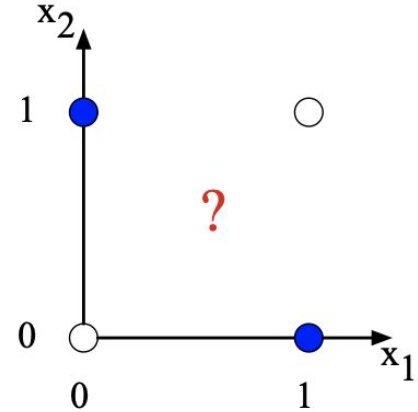
Decision Boundaries



a) x_1 AND x_2



b) x_1 OR x_2



c) x_1 XOR x_2

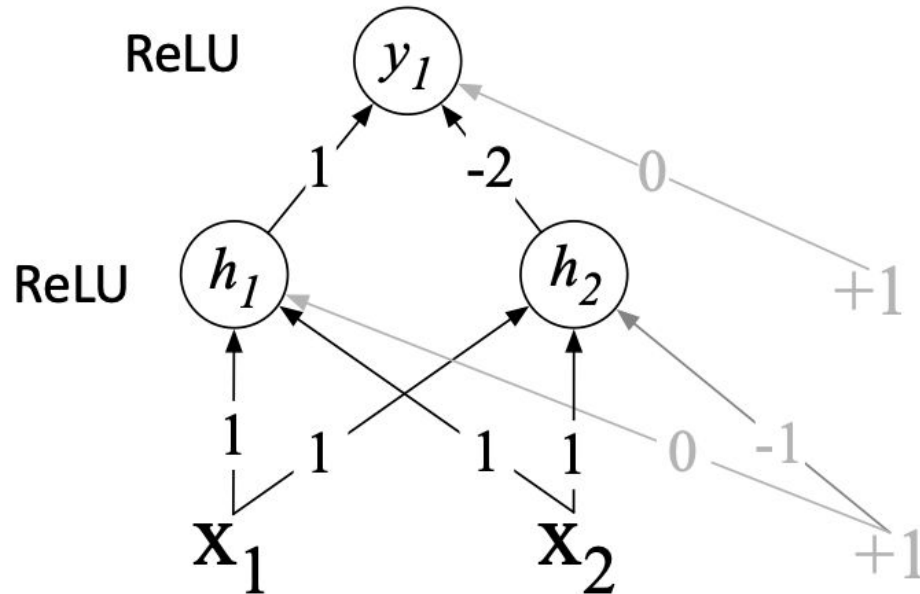
XOR is not a **linearly separable** function!

Solution to the XOR problem

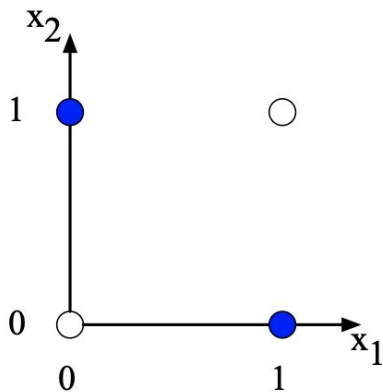
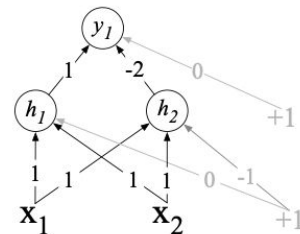
XOR **can't** be calculated by a single perceptron

XOR **can** be calculated by a layered network of units.

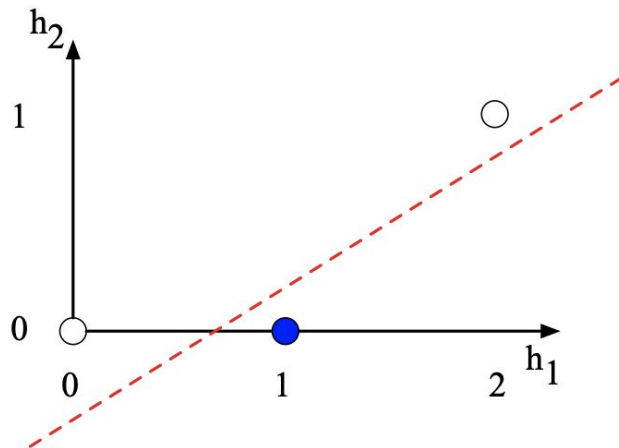
XOR		
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0



The hidden representation h



a) The original x space

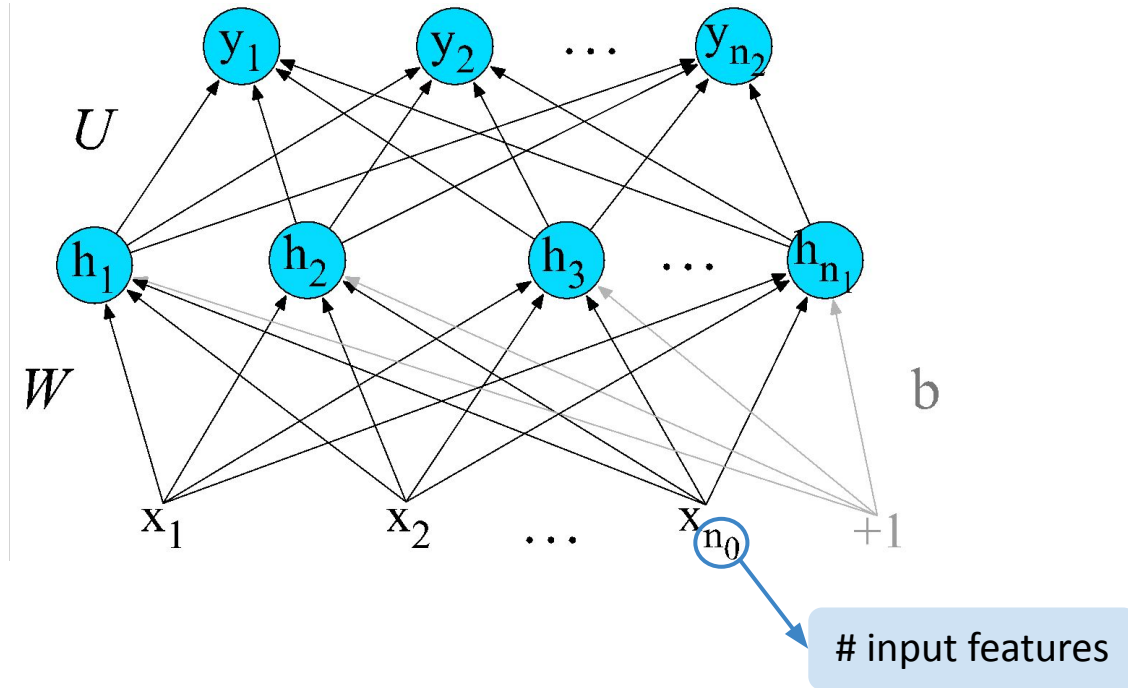


b) The new (linearly separable) h space

(With learning: hidden layers will learn to form useful representations)

Feedforward Neural Networks

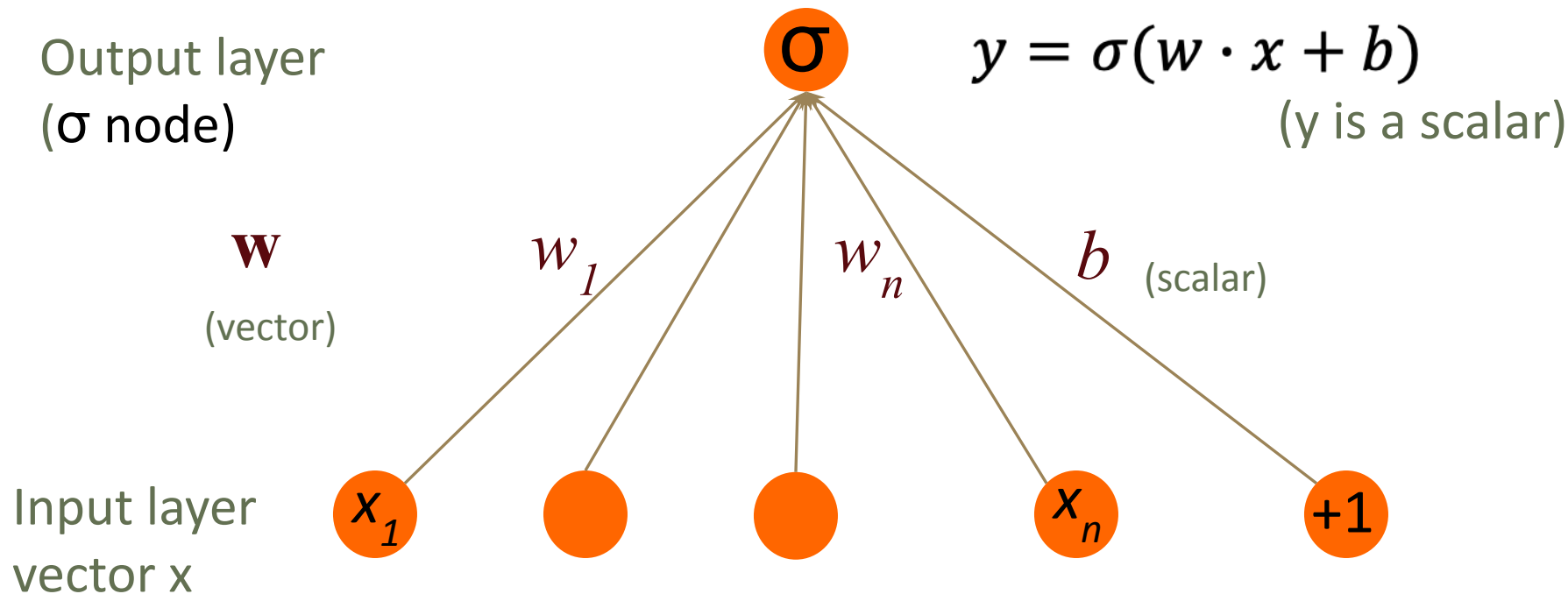
- Historically, also called **multilayer perceptrons**



Let's revisit logistic regression

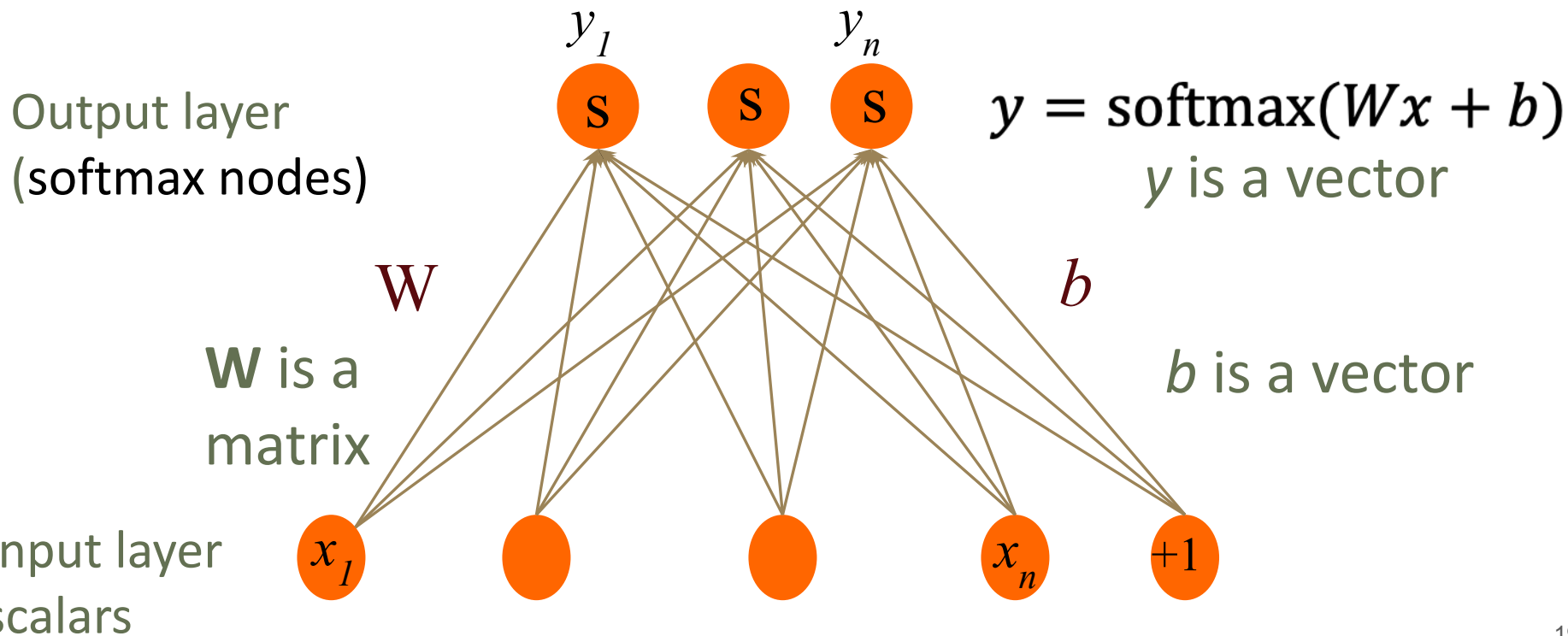
Binary Logistic Regression as a 1-Layer Network

(we don't count the input layer in counting layers!)

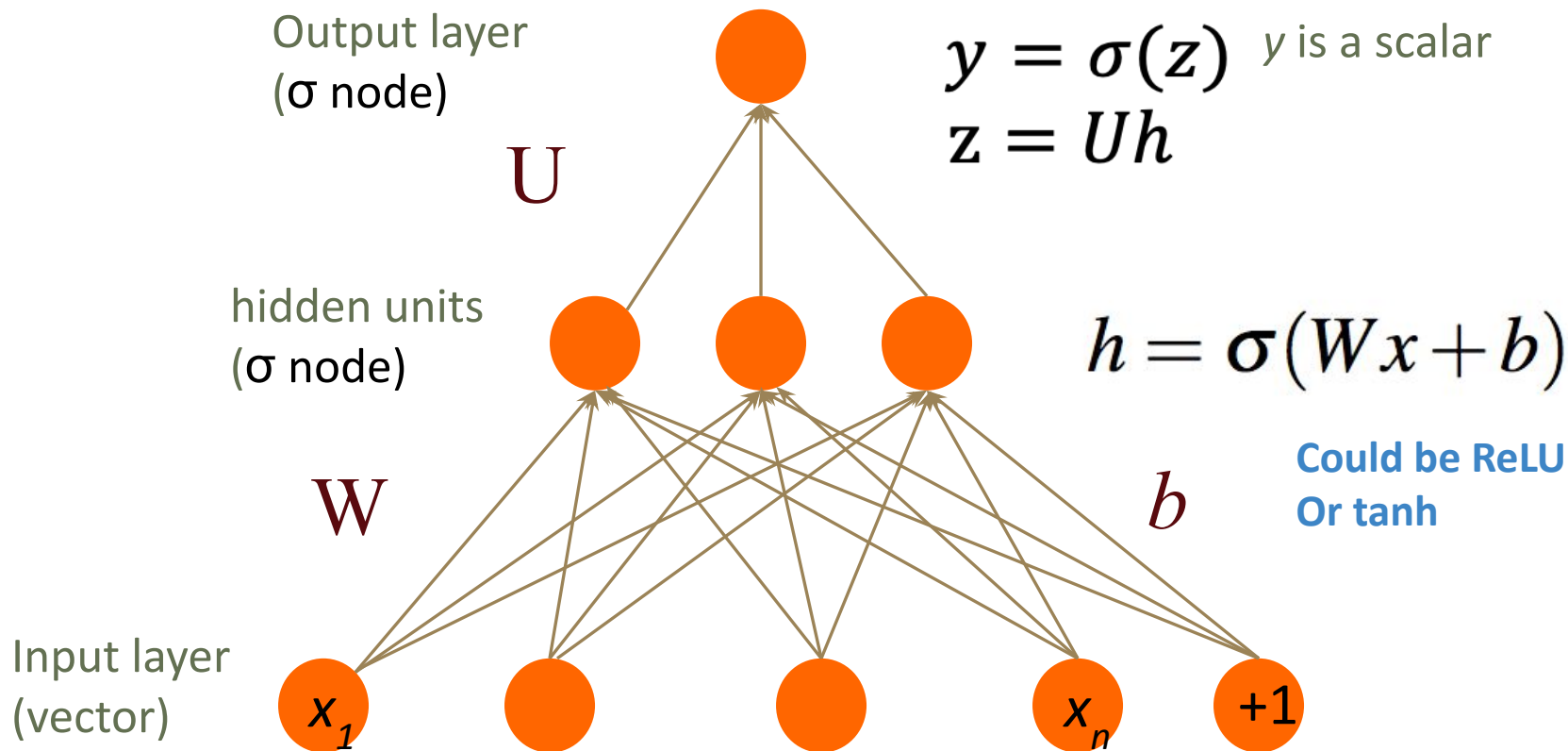


Multinomial Logistic Regression as a 1-Layer Network

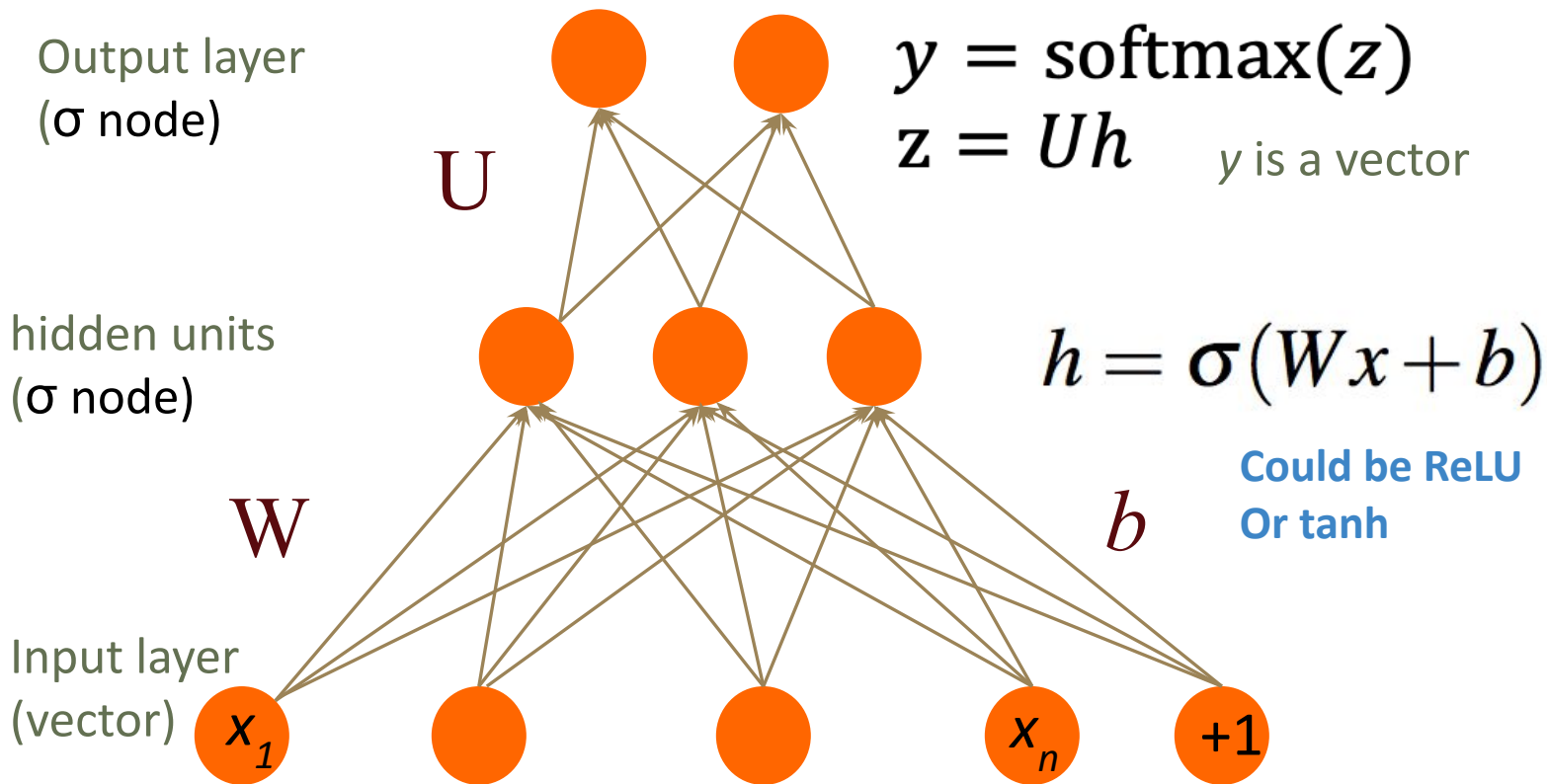
Fully connected single layer network



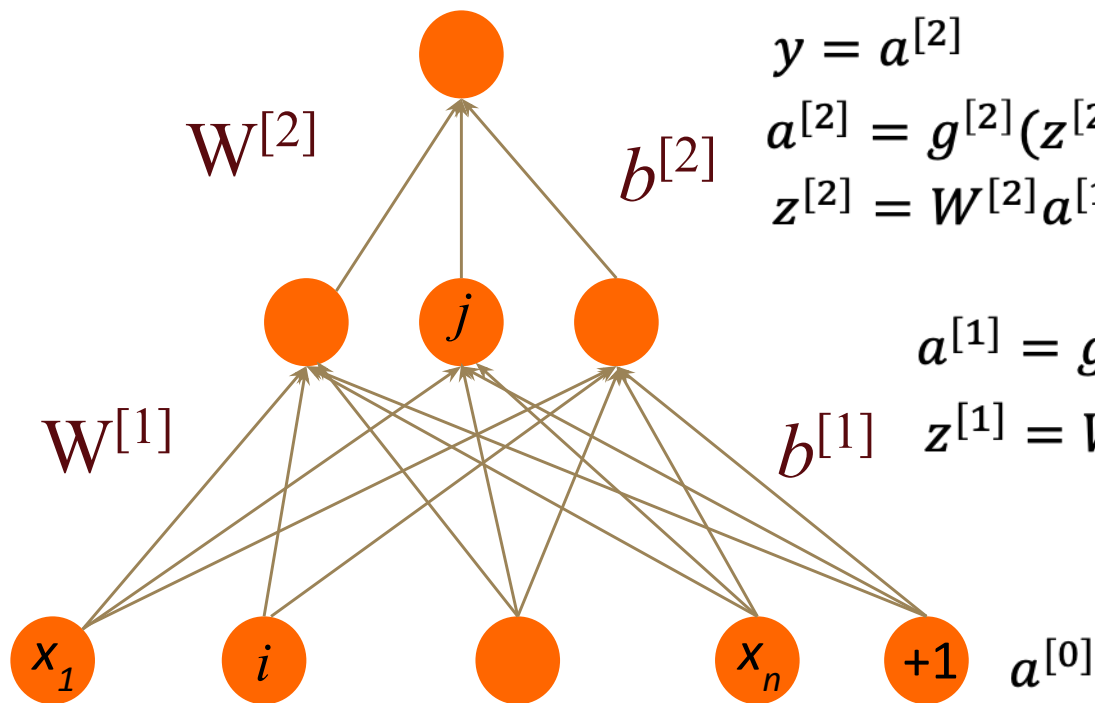
Two-Layer Network with Scalar Output



Two-Layer Network with Softmax Output



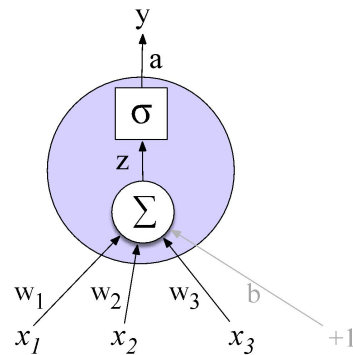
Multi-Layer FFNN: Notation



$$y = a^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]}) \quad \text{sigmoid or softmax}$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$



$$a^{[1]} = g^{[1]}(z^{[1]}) \quad \text{ReLU}$$

$$z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$$

for i in $1..n$

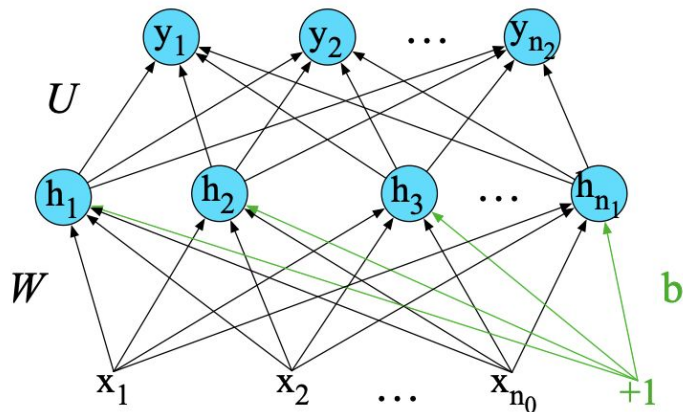
$$z^{[i]} = W^{[i]} a^{[i-1]} + b^{[i]}$$

$$a^{[i]} = g^{[i]}(z^{[i]})$$

$$\hat{y} = a^{[n]}$$

Multi-Layer FFNN: Bias Notation

Instead of:

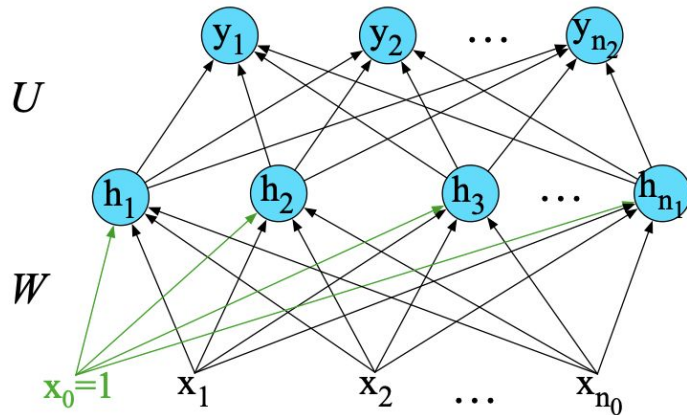


$$x = x_1, x_2, \dots, x_{n_0}$$

$$h = \sigma(Wx + b)$$

$$h_j = \sigma \left(\sum_{i=1}^{n_0} W_{ji} x_i + b_j \right)$$

We'll use:



$$x = x_0, x_1, x_2, \dots, x_{n_0}$$

$$h = \sigma(Wx)$$

$$\sigma \left(\sum_{i=0}^{n_0} W_{ji} x_i \right)$$

1. Add a dummy node $a_0=1$ to each layer
2. Its weight w_0 will be the bias
3. So input layer $a^{[0]}_0=1$,
 ◦ And $a^{[1]}_0=1, a^{[2]}_0=1, \dots$

DEMO: <https://playground.tensorflow.org/>

Let's Consider 2 Use Cases:

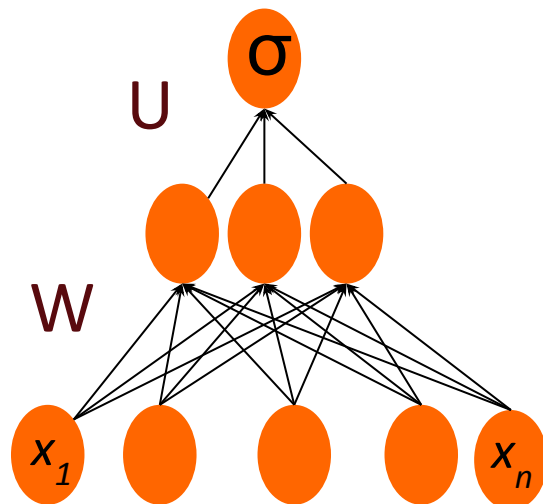
Text Classification
Language Modeling

FFNN: Classification

Classification: Sentiment Analysis

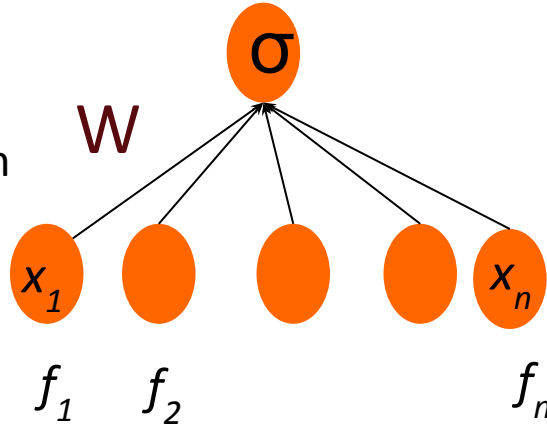
We could do exactly what we did with logistic regression:

Var	Definition
x_1	count(positive lexicon) $\in$ doc)
x_2	count(negative lexicon) $\in$ doc)
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_4	count(1st and 2nd pronouns $\in$ doc)
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_6	log(word count of doc)

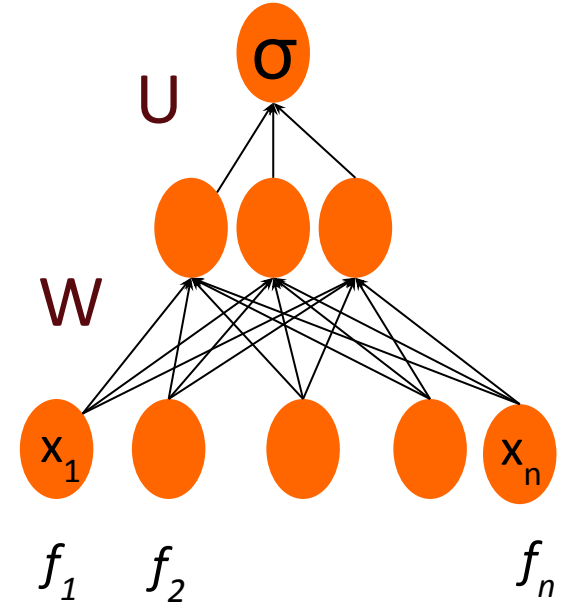


Feedforward Nets for Simple Classification

Logistic
Regression



2-layer
feedforward
network



Just adding a hidden layer to logistic regression:

- allows the network to use non-linear interactions between features
- which may (or may not) improve performance.

Reminder: Multiclass Outputs

What if you have more than two output classes?

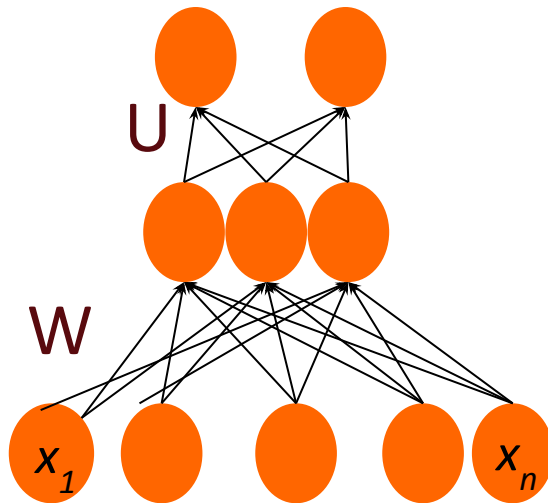
- Add more output units (one for each class)
- And use a “softmax layer”

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}} \quad 1 \leq i \leq D$$

The Output Layer:

$\hat{y}$ could have two nodes (one each for positive and negative), or 3 nodes (positive, negative, neutral).

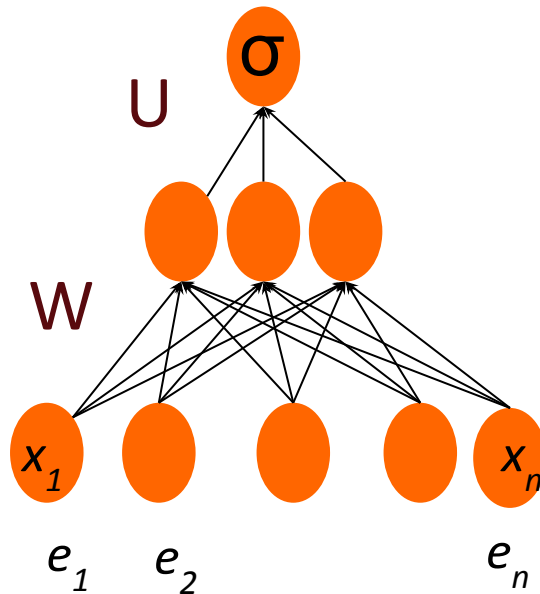
- $\hat{y}_1$ estimated probability of positive sentiment
- $\hat{y}_2$ probability of negative
- $\hat{y}_3$ probability of neutral



Even Better: Representation Learning

The real power of deep learning comes from the ability to **learn** features from the data

Instead of using hand-built
human-engineered features for classification
Use learned representations like
embeddings!



Embedding matrix $\mathbf{E}$

- An embedding is a vector of dimension $[1 \times d]$ that represents the input token.
- An embedding matrix $\mathbf{E}$ is a dictionary, one row per token of vocab V
- $\mathbf{E}$ has shape $[|V| \times d]$
- Embedding matrices are central to NLP; they represent input text in LLMs

Text Classification From Embeddings:

- Given tokenized input: dessert was great
- Select the embedding vectors from $\mathbf{E}$:
 1. Convert BPE tokens into vocabulary indices
 - $w = [3, 9824, 226]$
 2. Use indexing to select the corresponding rows from $\mathbf{E}$
 - row 3, row 4000, row 10532

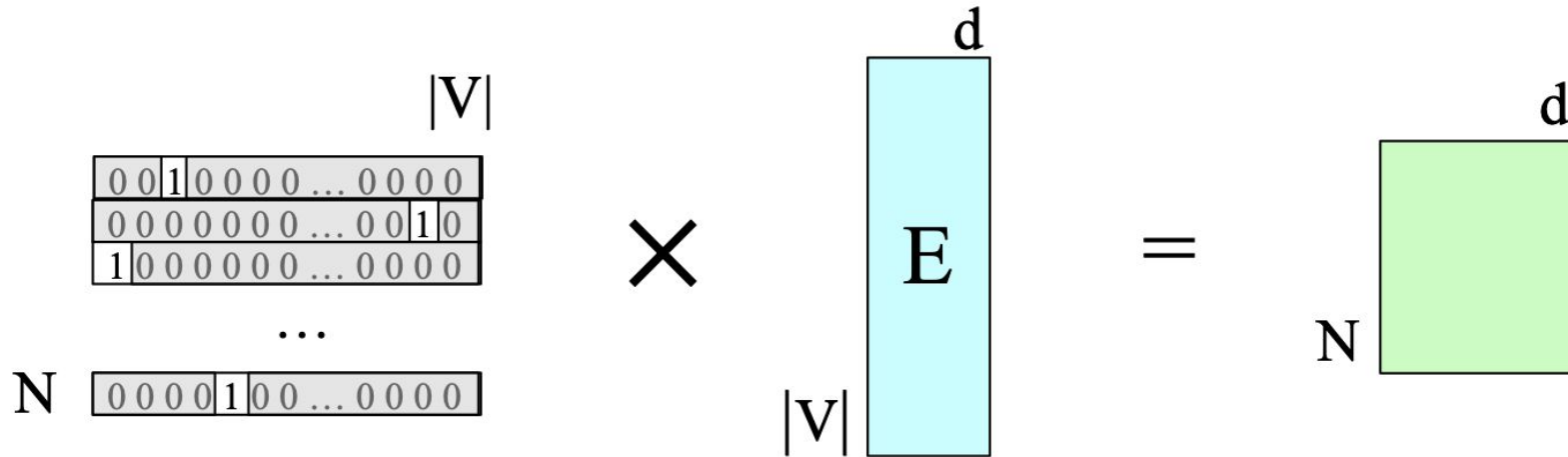
Another Way to Think of Indexing From E

- Treat each input word as one-hot vector
 - If **dessert** is index 3: $[0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ \dots \ 0 \ 0 \ 0 \ 0]$
1 2 3 4 5 6 7 $|V|$
- Multiply it by E to select out the embedding for dessert

The diagram illustrates the process of selecting an embedding from a matrix E using a one-hot vector. On the left, a one-hot vector of size $|V|$ is shown with a '1' at index 3 and zeros elsewhere. This vector is multiplied (indicated by a large $\times$) by an embedding matrix E of size $|V| \times d$. The matrix E has a row highlighted in green at index 3. The result of the multiplication is a single row vector of size d , also highlighted in green, representing the embedding for the word at index 3.

$$\begin{matrix} & 3 & & |V| \\ 1 & [0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ \dots \ 0 \ 0 \ 0 \ 0] \end{matrix} \times \begin{matrix} & d \\ 3 & \text{---} \\ & E \\ |V| & \text{---} \end{matrix} = \begin{matrix} & d \\ 1 & \text{---} \end{matrix}$$

Collecting embeddings from E for N words



Given window of N tokens, represented by $N [1 \times d]$ embeddings
Need to return a single class (e.g., pos or neg)

Text comes in different lengths

Given window of N tokens, represented by $N [1 \times d]$ embeddings, return a single class (e.g., pos or neg)

1. **Concatenate** all the inputs into one long vector of shape $[1 \times dN]$, i.e. input is length N of longest review
 - If shorter then pad with zero embeddings
 - Truncate if you get longer reviews at test time
2. **Pool** the inputs into a single short $[1 \times d]$ vector. A single "sentence embedding" (the same dimensionality as a word) to represent all the words. Less info, but very efficient and fast.

Pooling

Intuition: exact position not so important for sentiment.

We'll just do some sort of averaging of all the vectors.

Mean pooling:

$$\mathbf{x}_{mean} = \frac{1}{N} \sum_{i=1}^N \mathbf{e}(w_i)$$

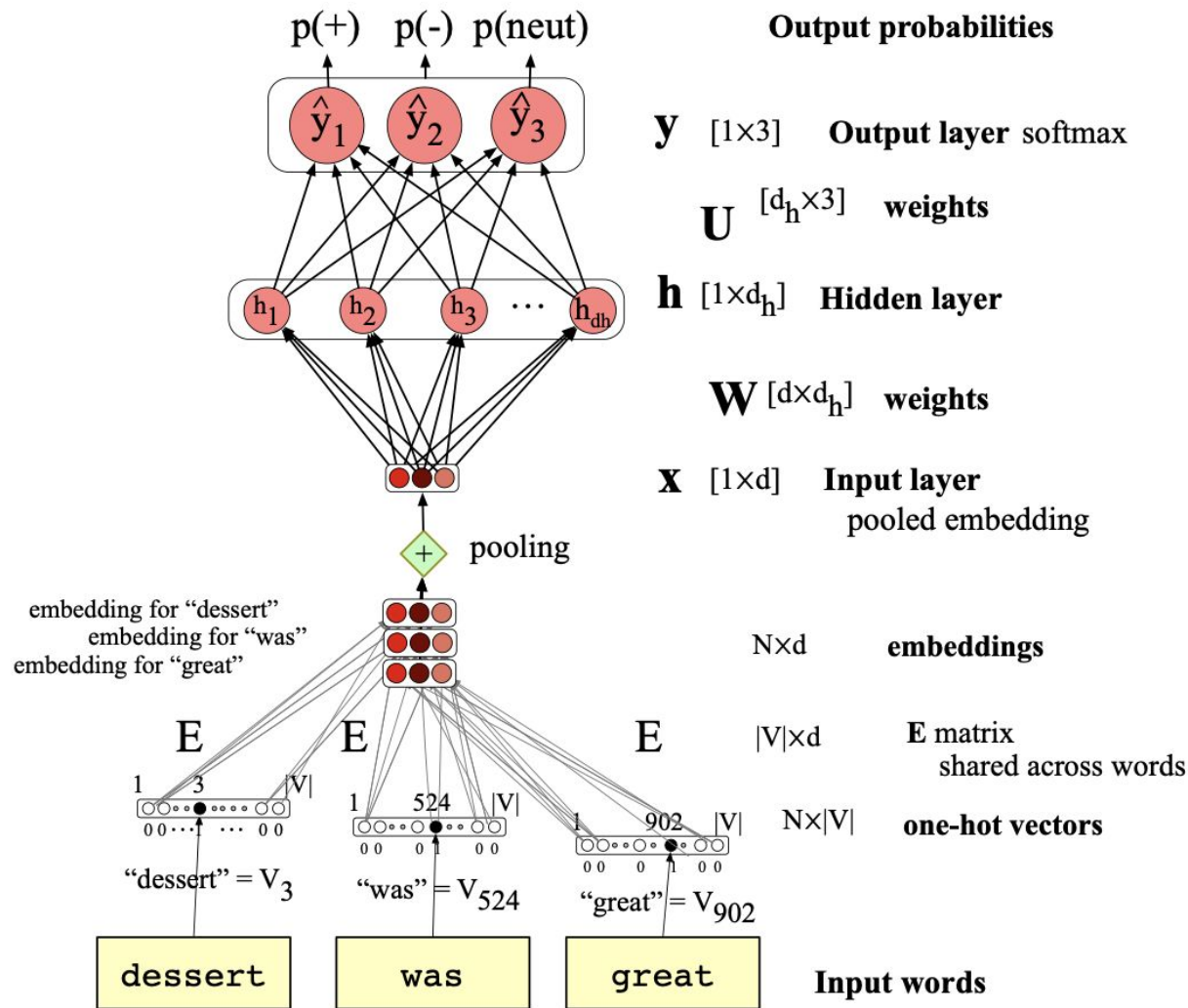
$$\mathbf{x} = \text{mean}(\mathbf{e}(w_1), \mathbf{e}(w_2), \dots, \mathbf{e}(w_n))$$

$$\mathbf{h} = \sigma(\mathbf{xW} + \mathbf{b})$$

$$\mathbf{z} = \mathbf{hU}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

Pooling



FFNN: Language Modeling

Neural Language Models

Language Modeling: Calculating the probability of the next word in a sequence given some history.

- We've seen N-gram based LMs
- But neural network LMs far outperform n-gram language models
- State-of-the-art neural LMs are based on more powerful neural network technology like Transformers
- Simple feedforward LMs introduce many important concepts

Simple feedforward Neural Language Models

Task: predict next word w_t

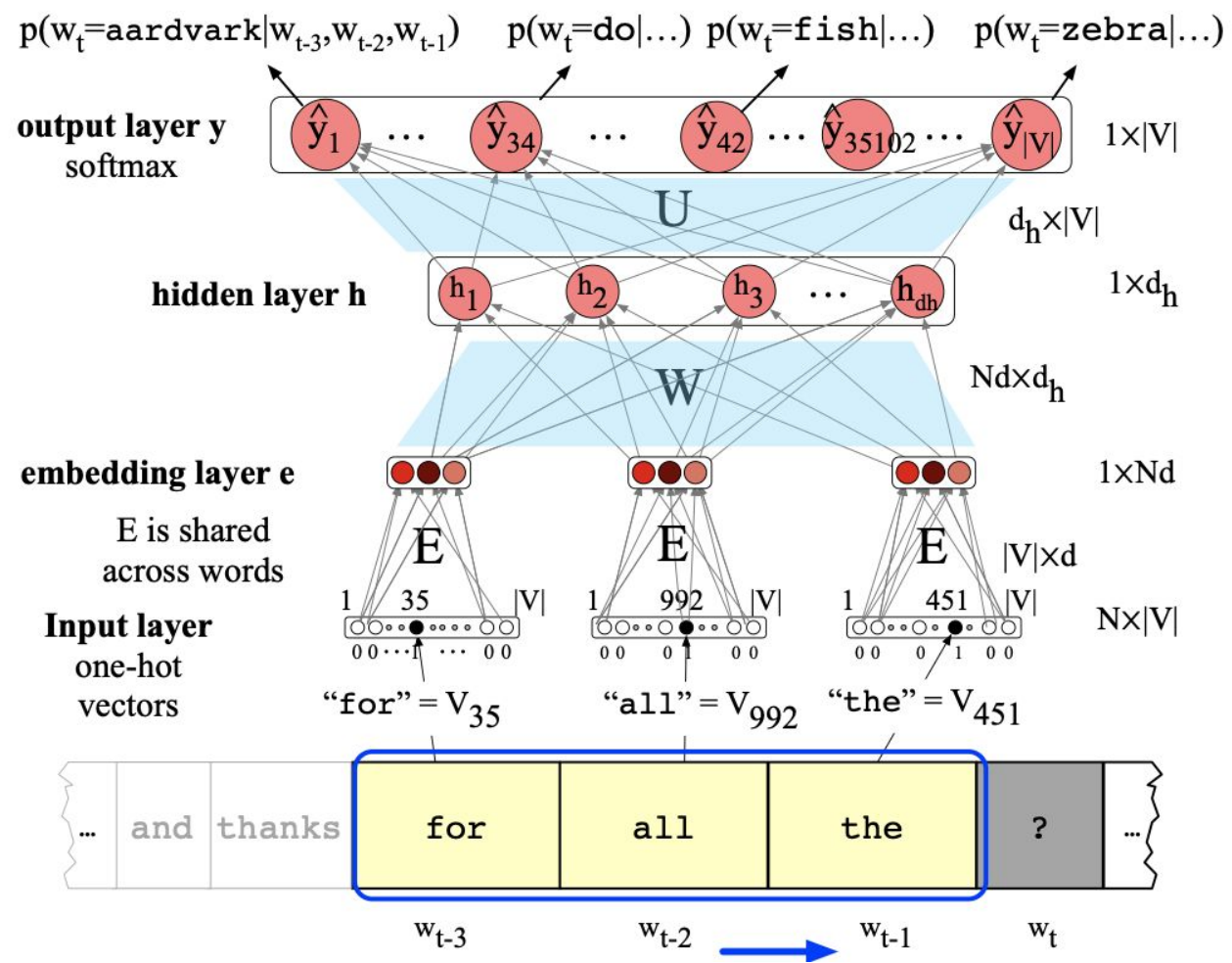
given prior words w_{t-1} w_{t-2} w_{t-3} $\dots$

Problem: Now we're dealing with sequences of arbitrary length.

Solution: Sliding windows (of fixed length)

$$P(w_t | w_1, \dots, w_{t-1}) \approx P(w_t | w_{t-N+1}, \dots, w_{t-1})$$

Inference in a Feedforward Language Model



Feedforward language model equations

$$\mathbf{e} = [\mathbf{Ex}_{t-3}; \mathbf{Ex}_{t-2}; \mathbf{Ex}_{t-1}]$$

$$\mathbf{h} = \sigma(\mathbf{We} + \mathbf{b})$$

$$\mathbf{z} = \mathbf{Uh}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

So $\hat{y}_{42}$ is the probability of the next word w_t being $V_{42} = \text{fish}$

Why Neural LMs work better than N-gram LMs

Training data:

We've seen: I have to make sure that the cat gets fed.

Never seen: dog gets fed

Test data:

I forgot to make sure that the dog gets ____

N-gram LM can't predict "fed"!

Neural LM can use similarity of "cat" and "dog" embeddings to generalize and predict "fed" after dog

TODOs:

- **Project Team Formation Due TODAY: 9/23**
- **Homework 1 Due: 9/30**
- **Project Proposal Due: 10/02**

Next Class: Backpropagation & RNNs